

- Description

commons-math-4.0-beta1-src\commons-math-
legacy\src\main\java\org\apache\commons\math4\legacy\analysis\solvers\
UnivariateSolverUtils.java

- Specification

module UnivariateSolverUtils

type

lower, upper, a, b, c: real

maxEval, maxIter: integer

initial, lowerBound, upperBound: real

outLower, outUpper: real

mid: real

ex: set of { None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations, NotANumber }

process isBracketing(f, lower, upper: real) r: boolean, ex: set of { None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations, NotANumber }

functional scenario form:

f = 0 && ex = NullArgument

|| hasNaN=0 && ex = NotANumber

|| isOrdered=0 && ex = BadSequence

|| bracketed=1 && ex = None && r = true

|| bracketed=0 && ex = None && r = false

end process

process verifyBracketing(f, lower, upper: real) ex: set of { None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations, NotANumber }

functional scenario form:

(f = null) && ex = NullArgument

OR (hasNaN(lower,upper)) && ex = NotANumber

OR (¬isOrdered(lower,upper)) && ex = BadSequence

OR (bracketed(yLower,yUpper)) && ex = None

OR (¬bracketed(yLower,yUpper)) && ex = NoBracketing

end process

process isSequence(a, b, c: real) r: boolean, ex: set of { None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations, NotANumber }

functional scenario form:

(hasNaN(a,b,c)) && ex = NotANumber

OR (a < b ∧ b < c) && ex = None && r = true

OR (¬(a < b ∧ b < c)) && ex = None && r = false

end process

process verifySequence(a, b, c: real) ex: set of { None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations, NotANumber }

functional scenario form:

(hasNaN(a,b,c)) && ex = NotANumber

OR (a < b $\wedge$ b < c) && ex = None

OR ($\neg$ (a < b $\wedge$ b < c)) && ex = BadSequence

end process

process verifyInterval(lower, upper: real) ex: set of { None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations, NotANumber }

functional scenario form:

(hasNaN(lower,upper)) && ex = NotANumber

OR (lower < upper) && ex = None

OR ($\neg$ (lower < upper)) && ex = BadSequence

end process

process midpoint(a, b) mid: real, ex

functional scenario form:

(hasNaN(a,b)) && ex = NotANumber

OR (true) && ex = None && mid = (a + b) / 2

end process

process bracket(f, initial, lowerBound, upperBound, maxIter) outLower: real, outUpper: real, ex

functional scenario form:

(f = null) && ex = NullArgument

OR (hasNaN(initial,lowerBound,upperBound)) && ex = NotANumber

OR ($\neg$ isOrdered(lowerBound, upperBound)) && ex = BadSequence

OR ($\neg$ (lowerBound $\leq$ initial $\wedge$ initial $\leq$ upperBound)) && ex = BadSequence

OR (yLower = f(lowerBound), yUpper = f(upperBound), bracketed(yLower, yUpper))

&& ex = None && outLower = lowerBound && outUpper = upperBound

OR ($\exists k \leq \text{maxIter} : \exists Lk, Uk : \text{lowerBound} \leq Lk < \text{initial} < Uk \leq \text{upperBound} \wedge \text{bracketed}(f(Lk), f(Uk))$) && ex = None && outLower = Lk && outUpper = Uk

OR (after maxIter expansions and $\forall L, U$ in [lowerBound, upperBound] around initial : $\neg \text{bracketed}(f(L), f(U))$) && ex = TooManyEvaluations

end process

end module

- Java Code

```
/*  
 * Licensed to the Apache Software Foundation (ASF) under one or more  
 * contributor license agreements. See the NOTICE file distributed with
```

```

* this work for additional information regarding copyright ownership.
* The ASF licenses this file to You under the Apache License, Version
2.0
* (the "License"); you may not use this file except in compliance with
* the License. You may obtain a copy of the License at
*
*     http://www.apache.org/licenses/LICENSE-2.0
*
* Unless required by applicable law or agreed to in writing, software
* distributed under the License is distributed on an "AS IS" BASIS,
* WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
* See the License for the specific language governing permissions and
* limitations under the License.
*/
package org.apache.commons.math4.legacy.analysis.solvers;
import org.apache.commons.math4.legacy.analysis.UnivariateFunction;
import org.apache.commons.math4.legacy.exception.NoBracketingException;
import org.apache.commons.math4.legacy.exception.NotStrictlyPositiveExc
eption;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;
import org.apache.commons.math4.legacy.exception.util.LocalizedFormats;
import org.apache.commons.math4.core.jdkmath.JdkMath;
/**
 * Utility routines for {@link UnivariateSolver} objects.
 *
 */
public final class UnivariateSolverUtils {
    /**
     * Class contains only static methods.
     */
    private UnivariateSolverUtils() {
    }
    /**
     * Convenience method to find a zero of a univariate real function. A
default
     * solver is used.
     *
     * @param function Function.
     * @param x0        Lower bound for the interval.
     * @param x1        Upper bound for the interval.
     * @return a value where the function is zero.

```

```

    * @throws NoBracketingException if the function has the same sign at
the
    *
        endpoints.
    * @throws NullArgumentException if {@code function} is {@code null}.
    */
    public static double solve(UnivariateFunction function, double x0, dou
ble x1)
        throws NullArgumentException, NoBracketingException {
        if (function == null) {
            throw new NullArgumentException(LocalizedFormats.FUNCTION);
        }
        final UnivariateSolver solver = new BrentSolver();
        return solver.solve(Integer.MAX_VALUE, function, x0, x1);
    }
    /**
    * Convenience method to find a zero of a univariate real function. A
default
    * solver is used.
    *
    * @param function      Function.
    * @param x0            Lower bound for the interval.
    * @param x1            Upper bound for the interval.
    * @param absoluteAccuracy Accuracy to be used by the solver.
    * @return a value where the function is zero.
    * @throws NoBracketingException if the function has the same sign at
the
    *
        endpoints.
    * @throws NullArgumentException if {@code function} is {@code null}.
    */
    public static double solve(UnivariateFunction function, double x0, dou
ble x1, double absoluteAccuracy)
        throws NullArgumentException, NoBracketingException {
        if (function == null) {
            throw new NullArgumentException(LocalizedFormats.FUNCTION);
        }
        final UnivariateSolver solver = new BrentSolver(absoluteAccuracy);
        return solver.solve(Integer.MAX_VALUE, function, x0, x1);
    }
    /**
    * Force a root found by a non-
bracketing solver to lie on a specified side, as
    * if the solver were a bracketing one.
    *

```

```

    * @param maxEval      maximal number of new evaluations of the func-
tion
    *                      (evaluations already done for finding the root
should
    *                      have already been subtracted from this number)
    * @param f            function to solve
    * @param bracketing    bracketing solver to use for shifting the roo
t
    * @param baseRoot      original root found by a previous non-
bracketing
    *                      solver
    * @param min           minimal bound of the search interval
    * @param max           maximal bound of the search interval
    * @param allowedSolution the kind of solutions that the root-
finding algorithm
    *                      may accept as solutions.
    * @return a root approximation, on the specified side of the exact ro
ot
    * @throws NoBracketingException if the function has the same sign at
the
    *                      endpoints.
    */
    public static double forceSide(final int maxEval, final UnivariateFunc-
tion f,
        final BracketedUnivariateSolver<UnivariateFunction> bracketing, fina
l double baseRoot, final double min,
        final double max, final AllowedSolution allowedSolution) throws NoBr
acketingException {
        if (allowedSolution == AllowedSolution.ANY_SIDE) {
            // no further bracketing required
            return baseRoot;
        }
        // find a very small interval bracketing the root
        final double step = JdkMath.max(bracketing.getAbsoluteAccuracy(),
            JdkMath.abs(baseRoot * bracketing.getRelativeAccuracy()));
        double xLo = JdkMath.max(min, baseRoot - step);
        double fLo = f.value(xLo);
        double xHi = JdkMath.min(max, baseRoot + step);
        double fHi = f.value(xHi);
        int remainingEval = maxEval - 2;
        while (remainingEval > 0) {
            if ((fLo >= 0 && fHi <= 0) || (fLo <= 0 && fHi >= 0)) {
                // compute the root on the selected side

```

```

    return bracketing.solve(remainingEval, f, xLo, xHi, baseRoot, allowedSolution);
}
// try increasing the interval
boolean changeLo = false;
boolean changeHi = false;
if (fLo < fHi) {
    // increasing function
    if (fLo >= 0) {
        changeLo = true;
    } else {
        changeHi = true;
    }
} else if (fLo > fHi) {
    // decreasing function
    if (fLo <= 0) {
        changeLo = true;
    } else {
        changeHi = true;
    }
} else {
    // unknown variation
    changeLo = true;
    changeHi = true;
}
// update the lower bound
if (changeLo) {
    xLo = JdkMath.max(min, xLo - step);
    fLo = f.value(xLo);
    remainingEval--;
}
// update the higher bound
if (changeHi) {
    xHi = JdkMath.min(max, xHi + step);
    fHi = f.value(xHi);
    remainingEval--;
}
}
throw new NoBracketingException(LocalizedFormats.FAILED_BRACKETING, xLo, xHi, fLo, fHi, maxEval - remainingEval, maxEval, baseRoot, min, max);
}
/**
 * This method simply calls

```

```

    * {@link #bracket(UnivariateFunction, double, double, double, double,
double, int)
    * bracket(function, initial, lowerBound, upperBound, q, r, maximumIte
rations)}
    * with {@code q} and {@code r} set to 1.0 and {@code maximumIteration
s} set to
    * {@code Integer.MAX_VALUE}.
    * <p>
    * <strong>Note: </strong> this method can take {@code Integer.MAX_VAL
UE}
    * iterations to throw a {@code ConvergenceException.} Unless you are
confident
    * that there is a root between {@code lowerBound} and {@code upperBou
nd} near
    * {@code initial}, it is better to use
    * {@link #bracket(UnivariateFunction, double, double, double, double,
double, int)
    * bracket(function, initial, lowerBound, upperBound, q, r, maximumIte
rations)},
    * explicitly specifying the maximum number of iterations.
    * </p>
    *
    * @param function    Function.
    * @param initial     Initial midpoint of interval being expanded to bra
cket a
    *                    root.
    * @param lowerBound  Lower bound (a is never lower than this value)
    * @param upperBound  Upper bound (b never is greater than this value).
    * @return a two-element array holding a and b.
    * @throws NoBracketingException      if a root cannot be bracketted.
    * @throws NotStrictlyPositiveException if {@code maximumIterations <=
0}.
    * @throws NullPointerException      if {@code function} is {@code n
ull}.
    */
    public static double[] bracket(UnivariateFunction function, double ini
tial, double lowerBound, double upperBound)
        throws NullPointerException, NotStrictlyPositiveException, NoBracke
tingException {
        return bracket(function, initial, lowerBound, upperBound, 1.0, 1.0, I
nteger.MAX_VALUE);
    }
    /**
    * This method simply calls

```

```

* {@link #bracket(UnivariateFunction, double, double, double, double,
double, int)}
* bracket(function, initial, lowerBound, upperBound, q, r, maximumIte
rations)}
* with {@code q} and {@code r} set to 1.0.
*
* @param function      Function.
* @param initial       Initial midpoint of interval being expanded
to
*                      bracket a root.
* @param lowerBound    Lower bound (a is never lower than this val
ue).
* @param upperBound    Upper bound (b never is greater than this v
alue).
* @param maximumIterations Maximum number of iterations to perform
* @return a two element array holding a and b.
* @throws NoBracketingException if the algorithm fails to find
a and b
*                      satisfying the desired conditions.
* @throws NotStrictlyPositiveException if {@code maximumIterations <=
0}.
* @throws NullPointerException if {@code function} is {@code n
ull}.
*/
public static double[] bracket(UnivariateFunction function, double ini
tial, double lowerBound, double upperBound,
    int maximumIterations) throws NullPointerException, NotStrictlyPosi
tiveException, NoBracketingException {
    return bracket(function, initial, lowerBound, upperBound, 1.0, 1.0, m
aximumIterations);
}
/**
* This method attempts to find two values a and b satisfying
* <ul>
* <li>{@code lowerBound <= a < initial < b <= upperBound}</li>
* <li>{@code f(a) * f(b) <= 0}</li>
* </ul>
* If {@code f} is continuous on {@code [a,b]}, this means that {@code
a} and
* {@code b} bracket a root of {@code f}.
* <p>
* The algorithm checks the sign of  $f(l_k)$  and  $f(u_k)$  for
increasing

```

```

* values of k, where  $l_k = \max(\text{lower}, \text{initial} - \delta_k)$ ,  $u_k =$ 
*  $\min(\text{upper}, \text{initial} + \delta_k)$ , using recurrence  $\delta_{k+1} = r$ 
*  $\delta_k + q$ ,  $\delta_0 = 0$  and starting search with  $k=1$ . The
e
* algorithm stops when one of the following happens:
* <ul>
* <li>at least one positive and one negative value have been found --
* success!</li>
* <li>both endpoints have reached their respective limits --
* NoBracketingException</li>
* <li>{@code maximumIterations} iterations elapse -
- NoBracketingException</li>
* </ul>
* <p>
* If different signs are found at first iteration ({@code k=1}), then
the
* returned interval will be  $[a, b] = [l_1, u_1]$ . If different s
igns are
* found at a later iteration {@code k>1}, then the returned interval
will be
* either  $[a, b] = [l_{k+1}, l_k]$  or  $[a, b] = [u_k, u_{k+1}]$ . A
* root solver called with these parameters will therefore start with
the
* smallest bracketing interval known at this step.
* </p>
* <p>
* Interval expansion rate is tuned by changing the recurrence paramet
ers
* {@code r} and {@code q}. When the multiplicative factor {@code r} i
s set to
* 1, the sequence is a simple arithmetic sequence with linear increas
e. When
* the multiplicative factor {@code r} is larger than 1, the sequence
has an
* asymptotically exponential rate. Note that the additive parameter {
{@code q}
* should never be set to zero, otherwise the interval would degenerat
e to the
* single initial point for all values of {@code k}.
* </p>
* <p>

```

```

* As a rule of thumb, when the location of the root is expected to be
* approximately known within some error margin, {@code r} should be s
et to 1
* and {@code q} should be set to the order of magnitude of the error
margin.
* When the location of the root is really a wild guess, then {@code r
} should
* be set to a value larger than 1 (typically 2 to double the interval
length at
* each iteration) and {@code q} should be set according to half the i
nitial
* search interval length.
* </p>
* <p>
* As an example, if we consider the trivial function {@code f(x) = 1
- x} and
* use {@code initial = 4}, {@code r = 1}, {@code q = 2}, the algorith
m will
* compute {@code f(4-2) = f(2) = -1} and {@code f(4+2) = f(6) = -
5} for
* {@code k = 1}, then {@code f(4-
4) = f(0) = +1} and {@code f(4+4) = f(8) = -7}
* for {@code k = 2}. Then it will return the interval {@code [0, 2]}
as the
* smallest one known to be bracketing the root. As shown by this exam
ple, the
* initial value (here {@code 4}) may lie outside of the returned brac
keting
* interval.
* </p>
*
* @param function      function to check
* @param initial       Initial midpoint of interval being expanded
to
*                      bracket a root.
* @param lowerBound    Lower bound (a is never lower than this val
ue).
* @param upperBound    Upper bound (b never is greater than this v
alue).
* @param q             additive offset used to compute bounds seque
nce
*                      (must be strictly positive)
* @param r             multiplicative factor used to compute bounds
sequence

```

```

* @param maximumIterations Maximum number of iterations to perform
* @return a two element array holding the bracketing values.
* @exception NoBracketingException if function cannot be bracketed in
the
*
* search interval
*/
public static double[] bracket(final UnivariateFunction function, final double initial, final double lowerBound,
    final double upperBound, final double q, final double r, final int maximumIterations)
    throws NoBracketingException {
    if (function == null) {
        throw new NullPointerException(LocalizedFormats.FUNCTION);
    }
    if (q <= 0) {
        throw new NotStrictlyPositiveException(q);
    }
    if (maximumIterations <= 0) {
        throw new NotStrictlyPositiveException(LocalizedFormats.INVALID_MAX_ITERATIONS, maximumIterations);
    }
    verifySequence(lowerBound, initial, upperBound);
    // initialize the recurrence
    double a = initial;
    double b = initial;
    double fa = Double.NaN;
    double fb = Double.NaN;
    double delta = 0;
    for (int numIterations = 0; numIterations < maximumIterations
        && (a > lowerBound || b < upperBound); ++numIterations) {
        final double previousA = a;
        final double previousFa = fa;
        final double previousB = b;
        final double previousFb = fb;
        delta = r * delta + q;
        a = JdkMath.max(initial - delta, lowerBound);
        b = JdkMath.min(initial + delta, upperBound);
        fa = function.value(a);
        fb = function.value(b);
        if (numIterations == 0) {
            // at first iteration, we don't have a previous interval
            // we simply compare both sides of the initial interval
            if (fa * fb <= 0) {
                // the first interval already brackets a root

```

```

        return new double[] { a, b };
    }
} else {
    // we have a previous interval with constant sign and expand it,
    // we expect sign changes to occur at boundaries
    if (fa * previousFa <= 0) {
        // sign change detected at near lower bound
        return new double[] { a, previousA };
    } else if (fb * previousFb <= 0) {
        // sign change detected at near upper bound
        return new double[] { previousB, b };
    }
}
}
// no bracketing found
throw new NoBracketingException(a, b, fa, fb);
}
/**
 * Compute the midpoint of two values.
 *
 * @param a a first value.
 * @param b second value.
 * @return the midpoint.
 */
public static double midpoint(double a, double b) {
    return (a + b) * 0.5;
}
/**
 * Check whether the interval bounds bracket a root. That is, if the v
alues at
 * the endpoints are not equal to zero, then the function takes opposi
te signs
 * at the endpoints.
 *
 * @param function Function.
 * @param lower Lower endpoint.
 * @param upper Upper endpoint.
 * @return {@code true} if the function values have opposite signs at
the given
 * points.
 * @throws NullPointerException if {@code function} is {@code null}.
 */
public static boolean isBracketing(UnivariateFunction function, final
double lower, final double upper)

```

```

throws NullPointerException {
    if (function == null) {
        throw new NullPointerException(LocalizedFormats.FUNCTION);
    }
    final double fLo = function.value(lower);
    final double fHi = function.value(upper);
    return (fLo >= 0 && fHi <= 0) || (fLo <= 0 && fHi >= 0);
}

/**
 * Check whether the arguments form a (strictly) increasing sequence.
 *
 * @param start First number.
 * @param mid Second number.
 * @param end Third number.
 * @return {@code true} if the arguments form an increasing sequence.
 */
public static boolean isSequence(final double start, final double mid,
final double end) {
    return start < mid && mid < end;
}

/**
 * Check that the endpoints specify an interval.
 *
 * @param lower Lower endpoint.
 * @param upper Upper endpoint.
 * @throws NumberIsTooLargeException if {@code lower >= upper}.
 */
public static void verifyInterval(final double lower, final double upper) throws NumberIsTooLargeException {
    if (lower >= upper) {
        throw new NumberIsTooLargeException(LocalizedFormats.ENDPOINTS_NOT_A
N_INTERVAL, lower, upper, false);
    }
}

/**
 * Check that {@code lower < initial < upper}.
 *
 * @param lower Lower endpoint.
 * @param initial Initial value.
 * @param upper Upper endpoint.
 * @throws NumberIsTooLargeException if {@code lower >= initial} or
 * {@code initial >= upper}.
 */

```

```

    public static void verifySequence(final double lower, final double initial, final double upper)
        throws NumberIsTooLargeException {
        verifyInterval(lower, initial);
        verifyInterval(initial, upper);
    }
    /**
     * Check that the endpoints specify an interval and the end points bracket a
     * root.
     *
     * @param function Function.
     * @param lower Lower endpoint.
     * @param upper Upper endpoint.
     * @throws NoBracketingException if the function has the same sign at the
     * endpoints.
     * @throws NullPointerException if {@code function} is {@code null}.
     */
    public static void verifyBracketing(UnivariateFunction function, final double lower, final double upper)
        throws NullPointerException, NoBracketingException {
        if (function == null) {
            throw new NullPointerException(LocalizedFormats.FUNCTION);
        }
        verifyInterval(lower, upper);
        if (!isBracketing(function, lower, upper)) {
            throw new NoBracketingException(lower, upper, function.value(lower), function.value(upper));
        }
    }
}
//VT
class USUVT {
    private static void pass(String name, boolean ok) {
        System.out.printf("[%s] %s%n", ok ? "PASS" : "FAIL", name);
    }
    // Shorthand aliases
    private static boolean isBrack(org.apache.commons.math4.legacy.analysis.UnivariateFunction f, double a, double b) {
        return org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.isBracketing(f, a, b);
    }
}

```

```

private static void verifyBrack(org.apache.commons.math4.legacy.analysis.UnivariateFunction f, double a, double b) {
    org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.verifyBracketing(f, a, b);
}
private static boolean isSeq(double a, double b, double c) {
    return org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.isSequence(a, b, c);
}
private static void verifySeq(double a, double b, double c) {
    org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.verifySequence(a, b, c);
}
private static void verifyInterval(double a, double b) {
    org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.verifyInterval(a, b);
}
private static double midpoint(double a, double b) {
    return org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.midpoint(a, b);
}
public void Vtest() {
    System.out.println("=== UnivariateSolverUtils V-method tests ===");
    // VT1: isBracketing with f == null -> should throw (NullPointerException)
    System.out.println("---- VT1 isBracketing(null, ...) ----");
    try {
        isBrack(null, -1.0, 1.0);
        pass("VT1 should throw", false);
    } catch (IllegalArgumentException ok) {
        System.out.println("threw: " + ok.getClass().getSimpleName());
        pass("VT1 threw", true);
    } catch (Throwable t) {
        t.printStackTrace();
        pass("VT1 wrong exception", false);
    }
    // Common functions for later tests
    org.apache.commons.math4.legacy.analysis.UnivariateFunction fLinear = x -> x; // root at 0
    org.apache.commons.math4.legacy.analysis.UnivariateFunction fShifted = x -> x - 2.0; // sign change over [0,3]
    org.apache.commons.math4.legacy.analysis.UnivariateFunction fPositive = x -> x * x + 1.0; // >0 everywhere
    // VT2: isBracketing false (no sign change; both positive)
    System.out.println("---- VT2 isBracketing false ----");
}

```

```

try {
    boolean r = isBrack(fPositive, -3.0, 2.0);
    System.out.println("isBracketing: " + r);
    pass("VT2 expect false", r == false);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT2 unexpected throw", false);
}
// VT3: isBracketing true (opposite signs)
System.out.println("---- VT3 isBracketing true ----");
try {
    boolean r = isBrack(fShifted, 0.0, 3.0); // f(0)=-2, f(3)=1
    System.out.println("isBracketing: " + r);
    pass("VT3 expect true", r == true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT3 unexpected throw", false);
}
// VT4: isBracketing true when endpoint is exact root (yLower == 0)
System.out.println("---- VT4 isBracketing endpoint-root at lower ----");
");
try {
    boolean r = isBrack(fLinear, 0.0, 1.0); // f(0)=0
    System.out.println("isBracketing: " + r);
    pass("VT4 expect true", r == true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT4 unexpected throw", false);
}
// VT5: verifyBracketing throws when not bracketed
System.out.println("---- VT5 verifyBracketing no-bracket ----");
try {
    verifyBrack(fPositive, -2.0, 5.0);
    pass("VT5 should throw NoBracketing", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    pass("VT5 threw", true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT5 wrong exception", false);
}
// VT6: verifyBracketing passes on opposite signs
System.out.println("---- VT6 verifyBracketing OK ----");
try {

```

```

    verifyBrack(fShifted, 0.0, 3.0);
    pass("VT6 no throw", true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT6 should not throw", false);
}
// VT7: verifyBracketing passes when endpoint is a root
System.out.println("---- VT7 verifyBracketing endpoint root ----");
try {
    verifyBrack(fLinear, -1.0, 0.0);
    pass("VT7 no throw", true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT7 should not throw", false);
}
// VT8: isSequence true
System.out.println("---- VT8 isSequence true ----");
try {
    boolean r = isSeq(1.0, 2.0, 3.0);
    System.out.println("isSequence: " + r);
    pass("VT8 expect true", r == true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT8 unexpected throw", false);
}
// VT9: isSequence false
System.out.println("---- VT9 isSequence false ----");
try {
    boolean r = isSeq(2.0, 1.0, 3.0);
    System.out.println("isSequence: " + r);
    pass("VT9 expect false", r == false);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT9 unexpected throw", false);
}
// VT10: verifySequence OK
System.out.println("---- VT10 verifySequence OK ----");
try {
    verifySeq(0.0, 0.5, 1.0);
    pass("VT10 no throw", true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT10 should not throw", false);
}

```

```

// VT11: verifySequence bad -> throw
System.out.println("---- VT11 verifySequence bad ----");
try {
    verifySeq(0.0, 2.0, 1.0);
    pass("VT11 should throw BadSequence", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    pass("VT11 threw", true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT11 wrong exception", false);
}
// VT12: verifyInterval OK
System.out.println("---- VT12 verifyInterval OK ----");
try {
    verifyInterval(-3.0, -2.0);
    pass("VT12 no throw", true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT12 should not throw", false);
}
// VT13: verifyInterval bad -> throw (lower >= upper)
System.out.println("---- VT13 verifyInterval bad ----");
try {
    verifyInterval(1.0, 1.0);
    pass("VT13 should throw BadSequence", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    pass("VT13 threw", true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT13 wrong exception", false);
}
// VT14: midpoint nominal
System.out.println("---- VT14 midpoint ----");
try {
    double m = midpoint(1.0, 3.0);
    System.out.printf("midpoint=%.9f\n", m);
    pass("VT14 expect 2.0", Math.abs(m - 2.0) <= 1e-12);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT14 unexpected throw", false);
}
// VT15: isBracketing with NaN bound -> should throw

```

```

System.out.println("---- VT15 isBracketing NaN ----");
try {
    isBrack(fLinear, Double.NaN, 1.0);
    pass("VT15 should throw", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    pass("VT15 threw", true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT15 wrong exception", false);
}
// VT16: verifyBracketing with reversed bounds (min>max) -> throw
System.out.println("---- VT16 verifyBracketing reversed ----");
try {
    verifyBrack(fShifted, 3.0, 0.0);
    pass("VT16 should throw BadSequence", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    pass("VT16 threw", true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT16 wrong exception", false);
}
// VT17: isSequence with NaN -> should throw (if implementation check
s); accept
// either throw or false
System.out.println("---- VT17 isSequence NaN ----");
try {
    boolean r = isSeq(Double.NaN, 1.0, 2.0);
    System.out.println("isSequence: " + r);
    // Some versions return false; others throw. Mark pass if false OR i
f thrown
    // above.
    pass("VT17 allow false", r == false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    pass("VT17 threw (acceptable)", true);
} catch (Throwable t) {
    t.printStackTrace();
    pass("VT17 unexpected", false);
}
// VT18: isBracketing true when endpoint is exact root at UPPER
System.out.println("---- VT18 isBracketing endpoint-root at upper ---
-");

```

```

    try {
        org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x - 3.0;
        boolean r = isBrack(f, 0.0, 3.0); // f(3)=0
        System.out.println("isBracketing: " + r);
        pass("VT18 expect true", r == true);
    } catch (Throwable t) {
        t.printStackTrace();
        pass("VT18 unexpected throw", false);
    }
    // VT19: isBracketing true when BOTH endpoints are roots (yLower=0 &&
yUpper=0)
    System.out.println("---- VT19 isBracketing both endpoints are roots -
---");
    try {
        org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
(x == 0.0 || x == 1.0) ? 0.0
        : (x - 0.5);
        boolean r = isBrack(f, 0.0, 1.0);
        System.out.println("isBracketing: " + r);
        pass("VT19 expect true", r == true);
    } catch (Throwable t) {
        t.printStackTrace();
        pass("VT19 unexpected throw", false);
    }
    // VT20: isBracketing with equal bounds (lower == upper) -> BadSequen
ce
    System.out.println("---- VT20 isBracketing equal bounds ----");
    try {
        isBrack(fShifted, 2.0, 2.0);
        pass("VT20 should throw BadSequence", false);
    } catch (IllegalArgumentException ok) {
        System.out.println("threw: " + ok.getClass().getSimpleName());
        pass("VT20 threw", true);
    } catch (Throwable t) {
        t.printStackTrace();
        pass("VT20 wrong exception", false);
    }
    System.out.println("=== Done UnivariateSolverUtils V-
method tests ===");
}
}
// FT
class USUFT {

```

```

private static boolean isBrack(org.apache.commons.math4.legacy.analysis.UnivariateFunction f, double a, double b) {
    return org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.isBracketing(f, a, b);
}

private static void verifyBrack(org.apache.commons.math4.legacy.analysis.UnivariateFunction f, double a, double b) {
    org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.verifyBracketing(f, a, b);
}

private static boolean isSeq(double a, double b, double c) {
    return org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.isSequence(a, b, c);
}

private static void verifySeq(double a, double b, double c) {
    org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.verifySequence(a, b, c);
}

private static void verifyInterval(double a, double b) {
    org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.verifyInterval(a, b);
}

private static double midpoint(double a, double b) {
    return org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolverUtils.midpoint(a, b);
}

private static void runIsBrack(String name, org.apache.commons.math4.legacy.analysis.UnivariateFunction f, double a,
    double b) {
    System.out.println("---
- " + name + " isBracketing a=" + a + " b=" + b + " ----");
    try {
        boolean r = isBrack(f, a, b);
        System.out.println("result=" + r);
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

private static void runVerifyBrack(String name, org.apache.commons.math4.legacy.analysis.UnivariateFunction f,
    double a, double b) {
    System.out.println("---
- " + name + " verifyBracketing a=" + a + " b=" + b + " ----");

```

```

    try {
        verifyBrack(f, a, b);
        System.out.println("ok");
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

private static void runIsSeq(String name, double a, double b, double c
) {
    System.out.println("---
- " + name + " isSequence a=" + a + " b=" + b + " c=" + c + " ----");
    try {
        boolean r = isSeq(a, b, c);
        System.out.println("result=" + r);
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

private static void runVerifySeq(String name, double a, double b, doub
le c) {
    System.out.println("---
- " + name + " verifySequence a=" + a + " b=" + b + " c=" + c + " ----
");
    try {
        verifySeq(a, b, c);
        System.out.println("ok");
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

private static void runVerifyInterval(String name, double a, double b)
{
    System.out.println("---
- " + name + " verifyInterval a=" + a + " b=" + b + " ----");
    try {
        verifyInterval(a, b);
        System.out.println("ok");
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

```

```

}
private static void runMidpoint(String name, double a, double b) {
    System.out.println("---
- " + name + " midpoint a=" + a + " b=" + b + " ----");
    try {
        double m = midpoint(a, b);
        System.out.printf("midpoint=%.17g%n", m);
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}
public void Ftest() {
    System.out.println("=== Fuzzing===");
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f1 = x ->
x - 3.172399649757028E307;
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f2 = x ->
x + 1.450546694021716E308;
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f3 = x ->
x + 2.608417489800224E307;
    // 1)
    runIsBrack("FT01", f1, 2.9000000000000001E307, 3.3000000000000006E307)
;
    // 2)
    runIsBrack("FT02", f2, -1.7976931348623157E308, -
1.2000000000000000E308);
    // 3)
    runIsBrack("FT03", f3, 3.000000000000000E307, -
3.500000000000000E307);
    // 4)
    runVerifyBrack("FT04", x -> x - (-1.7836486572462039E308), -
1.7976931348623157E308, -1.7000000000000000E308);
    // 5)
    runVerifyBrack("FT05", x -> x - 1.7836486572462039E308, 1.79000000000
00000E308, 1.7976931348623157E308);
    // 6)
    runIsSeq("FT06", -1.7976931348623157E308, -
2.608417489800224E307, 3.172399649757028E307);
    // 7)
    runIsSeq("FT07", -1.450546694021716E308, -1.450546694021716E308, -
1.0E307);
    // 8)
    runVerifySeq("FT08", -3.172399649757028E307, -
2.608417489800224E307, 1.7836486572462039E308);

```

```

// 9)
runVerifySeq("FT09", 1.7836486572462039E308, 1.7836486572462039E308,
1.7976931348623157E308);
// 10)
runVerifyInterval("FT10", -1.7836486572462039E308, -
1.7836486572462039E308);
// 11)
runVerifyInterval("FT11", -
2.608417489800224E307, 3.172399649757028E307);
// 12)
runMidpoint("FT12", 1.7976931348623157E308, -1.7976931348623157E308);
// 13)
runMidpoint("FT13", 1.7976931348623157E308, 1.7976931348623157E308);
// 14)
runIsBrack("FT14", x -> x - (-1.450546694021716E308), -
1.450546694021716E308, -1.0E308);
// 15)
runIsBrack("FT15", x -> x - 3.172399649757028E307, 1.0E307, 3.1723996
49757028E307);
// 16)
runVerifyBrack("FT16", x -> Math.abs(x) + 1.0, -
1.7976931348623157E308, 1.7976931348623157E308);
// 17)
runIsSeq("FT17", -
1.450546694021716E308, Double.POSITIVE_INFINITY, Double.POSITIVE_INFINI
TY);
// 18)
runVerifySeq("FT18", Double.NEGATIVE_INFINITY, -
1.0E308, Double.POSITIVE_INFINITY);
// 19)
runVerifyInterval("FT19", -0.0, +0.0);
// 20)
runVerifyBrack("FT20", x -> x + 1.0E307, -
1.7976931348623157E308, 3.172399649757028E307);
System.out.println("=== Done FUZZ ===");
}
}
//Z3
class USUZ3 {
private static void check(String name, boolean cond) {
System.out.printf("[%s] %s\n", cond ? "PASS" : "FAIL", name);
}
private static void runCase(String name, org.apache.commons.math4.lega
cy.analysis.UnivariateFunction f, double min,

```

```

        double max, int maxEval) {
System.out.println("---- " + name + " ----");
try {
    RiddersSolver solver = new RiddersSolver();
    double r = solver.solve(maxEval, f, min, max);
    System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
} catch (Throwable t) {
    System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
}
}

private static void runCaseWithStart(String name, org.apache.commons.m
ath4.legacy.analysis.UnivariateFunction f,
    double min, double max, double start, int maxEval) {
System.out.println("---- " + name + " (with start) ----");
try {
    RiddersSolver solver = new RiddersSolver();
    double r = solver.solve(maxEval, f, min, max, start);
    System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
} catch (Throwable t) {
    System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
}
}

private static void runCaseCustom(String name, org.apache.commons.math
4.legacy.analysis.UnivariateFunction f,
    double min, double max, int maxEval, double relAcc, double absAcc, d
ouble fAcc) {
System.out.println("---- " + name + " (custom tolerances) ----");
try {
//      RiddersSolver solver = new RiddersSolver(relAcc, absAcc, fAcc
);
//      double r = solver.solve(maxEval, f, min, max);
//      System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r)
);
} catch (Throwable t) {
    System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
}
}

public void Z3test() {
    System.out.println("Z3");
    // 1
    System.out.println("1: f=null -> NullArgument ");
}

```

```

try {
    new RiddersSolver().solve(100, null, -1.0, 1.0);
    check("S1 should throw NullArgument", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("S1 threw (NullArgument)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("S1 wrong exception", false);
}
// 2
System.out.println("2: min>max -> BadSequence");
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x - 1.0;
    new RiddersSolver().solve(100, f, 0.0, -0.5);
    check("S2 should throw BadSequence", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("S2 threw (BadSequence)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("S2 wrong exception", false);
}
// 3
System.out.println("3: not bracketed -> NoBracketing");
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x * x + 1.0;
    new RiddersSolver().solve(100, f, -999.0, 0.0);
    check("S3 should throw NoBracketing", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("S3 threw (NoBracketing)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("S3 wrong exception", false);
}
// 4
runCase("4: endpoint root at min (r=min)", x -> x, 0.0, 1.0, 200); //
f(0)=0
// 5
runCase("5: endpoint root at max (r=max)", x -> x - 1.0, 0.0, 1.0, 20
0); // f(1)=0

```

```

// 6
runCaseWithStart("6: start is root (r=start)", x -> x, -
1.0, 0.0, 0.0, 200);
// 7
runCaseWithStart("7: start*min opposite sign (expect r=0.5)", x -> x
- 0.5, 0.0, 2.0, 1.0, 200);
// 8
runCaseWithStart("8: start*max opposite sign (expect r=0.5)", x -> x
- 0.5, -1.0, 1.0, 0.0, 200);
// 9
runCaseCustom("9: r=xR=-1 with zero tolerances", x -> x + 1.0, -
2.0, 0.0, 200, 0.0, 0.0, 0.0);
// 10
runCase("10: narrowed bracket contains r=1.25", x -> x - 1.25, 0.0, 3
.0, 200);
// 11
System.out.println("11: TooManyEvaluations (maxEval=-1) ");
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
Math.cos(x) - x; // root ~0.739 in
                // [0,1]
    new RiddersSolver().solve(-1, f, 0.0, 1.0);
    check("11 should throw TooManyEvaluations", false);
} catch (IllegalStateException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("11 threw (TooManyEvaluations)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("11 wrong exception type", false);
}
// 12
runCase("12: endpoint root at max (r=max)", x -> x - 1.0, 0.0, 1.0, 2
00); // f(1)=0
// 13
runCaseWithStart("13: start is root (r=start)", x -> x, -
1.0, 0.0, 0.0, 200);
// 14
runCaseWithStart("14: start*min opposite sign (expect r=0.5)", x -> x
- 0.5, 0.0, 2.0, 1.0, 200);
// 15
runCaseWithStart("15: start*max opposite sign (expect r=0.5)", x -> x
- 0.5, -1.0, 1.0, 0.0, 200);
// 16

```

```

    runCaseCustom("16: r=xR=-1 with zero tolerances", x -> x + 1.0, -
2.0, 0.0, 200, 0.0, 0.0, 0.0);
    // 17
    runCase("17: endpoint root at max (r=max)", x -> x - 1.0, 0.0, 1.0, 2
00); // f(1)=0
    // 18
    runCaseWithStart("18: start is root (r=start)", x -> x, -
1.0, 0.0, 0.0, 200);
    // 19
    runCaseWithStart("19: start*min opposite sign (expect r=0.5)", x -> x
- 0.5, 0.0, 2.0, 1.0, 200);
    // 20
    runCaseWithStart("20: start*max opposite sign (expect r=0.5)", x -> x
- 0.5, -1.0, 1.0, 0.0, 200);
    System.out.println("=== Done ===");
}
}
class USUtest {
    public static void main(String[] args) {
        // VT
        USUVT t1 = new USUVT();
        t1.Vtest();
        // FT
        USUFT t2 = new USUFT();
        // t2.Ftest();
        // Z3
        USUZ3 t3 = new USUZ3();
        // t3.Z3test();
    }
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 25%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
UnivariateSolverUtils.java	31.0 %	26	58	84
USUZ3	0.0 %	0	2	2
UnivariateSolverUtils	25.0 %	17	51	68
isSequence(double, double, dou	75.0 %	3	1	4
verifyBracketing(UnivariateFunc	75.0 %	3	1	4
isBracketing(UnivariateFunction,	90.0 %	9	1	10
verifyInterval(double, double)	100.0 %	2	0	2
midpoint(double, double)		0	0	0
verifySequence(double, double,		0	0	0
USUVT	64.3 %	9	5	14
USUFT		0	0	0
USUtest		0	0	0

3. V-Method testing instruction coverage: 20.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ UnivariateSolverUtils.java	24.6 %	531	1,624	2,155
> USUZ3	0.6 %	3	497	500
> USUFT	0.6 %	3	472	475
▼ UnivariateSolverUtils	20.7 %	100	384	484
verifyBracketing(UnivariateFunc	76.7 %	23	7	30
isBracketing(UnivariateFunction,	100.0 %	37	0	37
isSequence(double, double, dou	100.0 %	12	0	12
midpoint(double, double)	100.0 %	6	0	6
verifyInterval(double, double)	100.0 %	15	0	15
verifySequence(double, double,	100.0 %	7	0	7
> USUVT	60.5 %	410	268	678
> USUtest	83.3 %	15	3	18

4. Fuzz testing branch coverage: 17.6%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ UnivariateSolverUtils.java	15.5 %	13	71	84
> USUVT	0.0 %	0	14	14
▼ UnivariateSolverUtils	17.6 %	12	56	68
isBracketing(UnivariateFunction,	70.0 %	7	3	10
verifyBracketing(UnivariateFunc	75.0 %	3	1	4
verifyInterval(double, double)	100.0 %	2	0	2
> USUZ3	50.0 %	1	1	2
> USUFT		0	0	0
> USUtest		0	0	0

5. Fuzz testing instruction coverage: 14%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ src/main/java	0.3 %	650	203,460	204,110
▼ org.apache.commons.math4.legacy.anal	4.7 %	650	13,091	13,741
RiddersSolver.java	8.4 %	120	1,311	1,431
▼ UnivariateSolverUtils.java	19.7 %	424	1,731	2,155
> USUVT	0.4 %	3	675	678
> USUFT	0.6 %	3	472	475
▼ UnivariateSolverUtils	14.0 %	68	416	484
verifyBracketing(UnivariateFunc	76.7 %	23	7	30
isBracketing(UnivariateFunction,	81.1 %	30	7	37
verifyInterval(double, double)	100.0 %	15	0	15
> USUZ3	67.0 %	335	165	500
> USUtest	83.3 %	15	3	18

6. Z3 branch coverage: 25%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ UnivariateSolverUtils.java	20.2 %	17	67	84
> USUVT	0.0 %	0	14	14
> USUZ3	0.0 %	0	2	2
▼ UnivariateSolverUtils	25.0 %	17	51	68
verifyBracketing(UnivariateFunc	75.0 %	3	1	4
isBracketing(UnivariateFunction,	80.0 %	8	2	10
isSequence(double, double, dou	100.0 %	4	0	4
verifyInterval(double, double)	100.0 %	2	0	2
midpoint(double, double)		0	0	0
verifySequence(double, double,		0	0	0
> USUFT		0	0	0
> USUtest		0	0	0

7. Z3 instruction coverage: 19.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ UnivariateSolverUtils.java	25.0 %	538	1,617	2,155
> USUVT	0.4 %	3	675	678
> USUZ3	0.6 %	3	497	500
▼ UnivariateSolverUtils	19.2 %	93	391	484
verifyBracketing(UnivariateFunc	76.7 %	23	7	30
isBracketing(UnivariateFunction,	81.1 %	30	7	37
isSequence(double, double, dou	100.0 %	12	0	12
midpoint(double, double)	100.0 %	6	0	6
verifyInterval(double, double)	100.0 %	15	0	15
verifySequence(double, double,	100.0 %	7	0	7
> USUtest	83.3 %	15	3	18
> USUFT	89.3 %	424	51	475